Activity 2, Advanced - Digitizing Drawings with SVG Code

Activity 2, Advanced - Digitizing Drawings with SVG Code

Download this activity

In this activity, students will get their hands on SVG code and learn how to render their physical Activity 1 drawings as digital illustrations. Students will follow the syntax available on BlindSVG.com to build their basic shapes using the attributes and values from the first activity, then get visual and tactile confirmation of their end result, getting to compare the output to their original drawings.

This activity will involve computer use, text editor navigation and text editing/input, being able to open the finished SVG files in Google Chrome or Mozilla Firefox, and outputting via any tactile method.

Setup - Activity 2

This activity can be broken down into several elements for lesson planning.

Objectives

- Begin to understand how to write SVG code and map physical drawings to the digital canvas space.
- Learn syntax, code and shape ordering, file setup, and output.
- Learn how to communicate with a visual interpreter when assessing accuracy of the digital drawing, and feel differences between tactile/embossed output and the original tactile drawings.
- Enjoy creating their first digital illustration from drawn concept to final digital result!

Materials Needed

- APH Draftsman Tactile Drawing Board or APH TactileDoodle
- Tactile Dot or Graph paper (Provided in TADA materials)

- Stylus or Pen
- Tactile Rulers
  - Cardstock or Cardboard/ for a straight edge that can be folded
- Laptop or Desktop computer
- Text Editor: TextEdit or Text Mate on Mac, Notepadd++ on Windows
- Google Chrome or Mozilla Firefox (Safari may not work for this process)
- Tactile Graphics output method: embosser, Swell-form, visual tracing method on standard printer output

Alternate Materials

- Sensational Blackboard

Ideas for Carrying Out This Activity

- Read through and have students read through BlindSVG.com, specifically the Home Page, Setup page, and the Basic Shapes page.
- Pay attention to good typing and syntax usage. Be ready to hunt through code to debug any missing spaces, quotation marks, or incorrect values.
- Use Swell-form if available for final output.
- If no other tactile output methods are available, print drawings out on basic printers and set print setting to flip the drawing horizontally so it prints out a mirror image. Place this print on a Draftsman and trace the resulting image with a pen for the student, embossing the image and making it appear exactly as intended when the paper is flipped around to expose the embossed lines for the student.

Adventure Map: Activity 2

Teaching tip: Provide sufficient time for the student to explore, develop skills, and have fun at each step! Encouraging creativity and personal preferences for drawing as much as possible. Some students may be able to accomplish each step in one session; most students will need several sessions to complete the adventure. This is a very technical part of the adventure, and this may become frustrating depending on the technical aptitude of the students and their screen reader/magnification skills. Please give them enough time and space to solve for technical bugs and typing errors; Google Chrome will do a basic debug of SVG code and tell the student what lines have errors when opening the SVG within the browser.

1. Introduction

Students will now get their first chance to translate their tactile physical drawing into an actual digital illustration using SVG code. This requires a good understanding of the attributes and values of their drawn shapes and comfort with manipulating and editing text in a basic text editor.

2. Read BlindSVG.com

To get prepared for this task, have students read through the following pages of BlindSVG.com:

- BlindSVG Home Page
- BlindSVG Setup Page
- BlindSVG Shapes Page

This will give them information about SVG coding, how to set up their first SVG text files, explain the overall concept of the SVG units (reinforced by Adventure 1), and give them syntax to create basic shapes.

3. Build Template SVG Files

- Have students open a new text file, then have them copy and paste the SVG file template code from the BlindSVG setup page.
- In the opening SVG tag, if the student has a landscape drawing, ensure they set the width attribute to 1100 and the height attribute to 850. Swap these numbers for a portrait orientation.
- Have the students save their files with the ".svg" extension instead of ".txt."

4. Identify Shapes and Start Coding!

- From the BlindSVG Basic Shapes page, have students identify the basic shapes they've drawn and match them to the shapes available on the website. Have them list out all the shapes they need, and proceed to have them copy and paste the correct shape code from the site into their text files.
- The students should match the attributes and values they came up with from Adventure 1 and input them into the shape code in their files. Students can copy and paste or can type out the syntax directly, whatever is most comfortable for them.
- Students should be taught that the order of the code matters. The last line of code before the ending `</svg>` tag will be the top-most shape layer in the drawing.
- **Thought experiment:** consider each line of shape code as the same shape cut out in construction paper. Each line of code they write is telling the computer to lay down another shape of construction paper on top of the last one. Pieces of the drawing that the students want in the background or behind other layers should be written first in the code, and everything in the foreground or appearing in front of other objects should come last in the code.

5. First Output and Debugging

- Once the students have all entered their shapes and verify that their attributes and values are all correct, have them open their SVG file in

Chrome or Firefox.

- Did Chrome print any errors on the screen? If so, identify the line the error has occurred on and have the student go back to their file and find and fix the error. This can be a missing or errant space, missing quotation mark, missing angle bracket, a missing slash to close out a shape, or a bad value/broken syntax.
- If the drawing renders, celebrate with the student!

6. Output and Tactile Assessment

- Send the rendered drawings to an embosser or use the flipped print and trace method to turn the digital file into a tangible tactile adventure for the student to explore. Have them compare their output to their original drawings:
  - Is anything out of place?
  - Does anything need tweaking?
  - Inspired to add more and iterate on the drawing?

7. Final tweaks

- Basic tactile output requires black outlines, white negative space, and can contain one or two additional colors for embossers that can produce different dot heights. These fill colors seem to work best with shades of gray or the CSS Extended Color Keyword "oldlace" or "seashell."
- Swell form will produce cleaner lines along with a visual print on top of the tactile print, so this can be used as a final output mechanism if available.
- Students can go back in and play with their code to tweak and iterate their final results. Give them visual interpretation now that they have a tangible piece of artwork to work from based on their original numbers.
- Do they have ideas of what more they could create now that they've built their first digital graphic?
- Does the syntax flow for them? Is it hard to understand or grasp?
- What would make it easier for them to understand the process?
- Does knowing that blind designers are actively using this process to make logos, art, technical drawings, layouts, engineering plans, circuit diagrams, charts and graphs, and more get them excited about the possibilities of this process?

Previous: Activity 1: Intro to Spatial Mapping · **Next:** Activity 3: Iterating on SVG Drawings & Beyond · **Back to:** Adventure Map · Download all TADA PDFs